



Факултет за информатички науки и компјутерско инженерство

Проектна задача

Предмет:

Вештачка интелигенција за игри

Тема:

Имплементација на Behavior Tree систем за
однесување на NPC агенти во stealth игра

Изработил:

Илија Бунчески – 221094

Ментор:

Проф. Д-р Андреа Кулаков

Скопје

Јуни, 2026



1. Вовед

SentryNode претставува stealth игра изработена за предметот „Вештачка интелигенција за игри“. Проектот прикажува како guard NPC може да донесува одлуки преку behavior tree, да користи визуелна и звучна перцепција, да патролира низ ниво со NavMesh и да реагира според постапките на играчот со состојби како сомнеж(suspicious), бркање(chasing), истрага(investigating) и пребарување(searching).

Целта не е да се направи целосна игра со многу содржина, туку јасен и читлив AI систем кој може да се демонстрира, анализира и надградува.

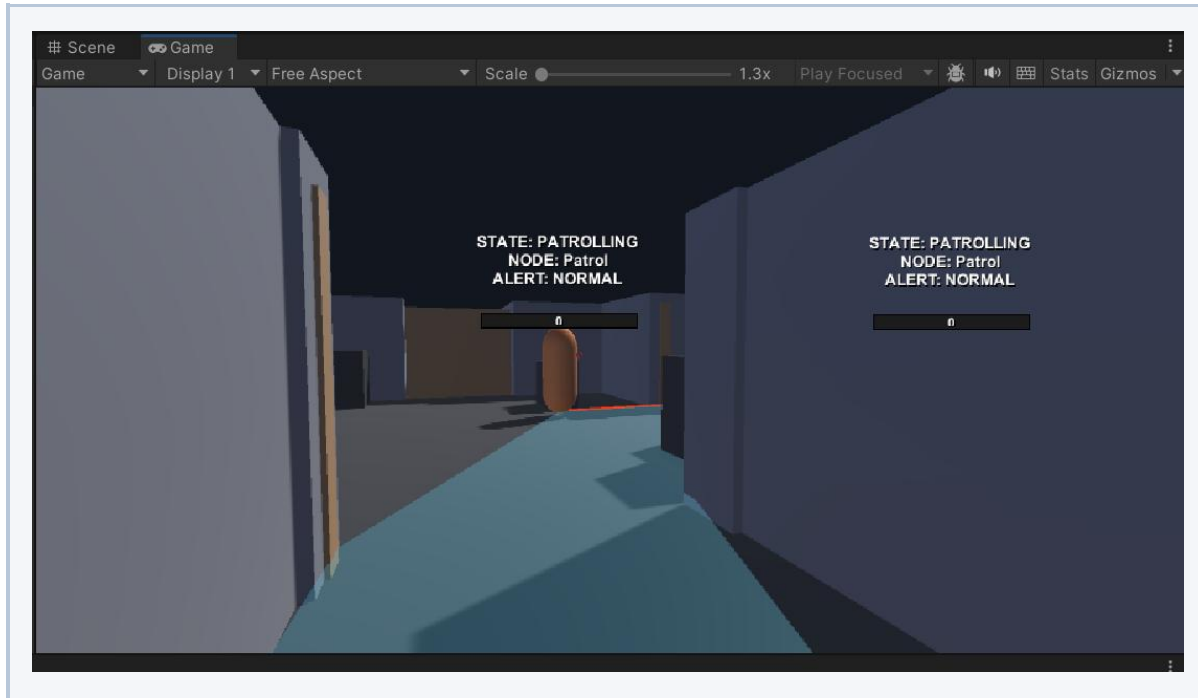
2. Основни информации

Unity верзија	2022.3.62f1
Тип	First-person stealth AI prototype
Навигација	Unity NavMesh и NavMeshAgent
Главни системи	Behavior tree, vision, hearing, patrol, search, alert и debug visualization

3. Играње на играта

Играчот се движи во first-person перспектива низ затворено демо ниво. Движењето е директно поврзано со stealth механиките: побрзото движење создава поголем шум, а crouch движењето е побавно и може да биде речиси тивко.

W/A/S/D	Движење
Mouse	Гледање и ротирање на камерата
Left Shift	Спринт
C	Клекнување
E	Интеракција – отворање врата



Слика: Движење на чувар

4. Stealth loop

Главниот gameplay циклус е следниот:

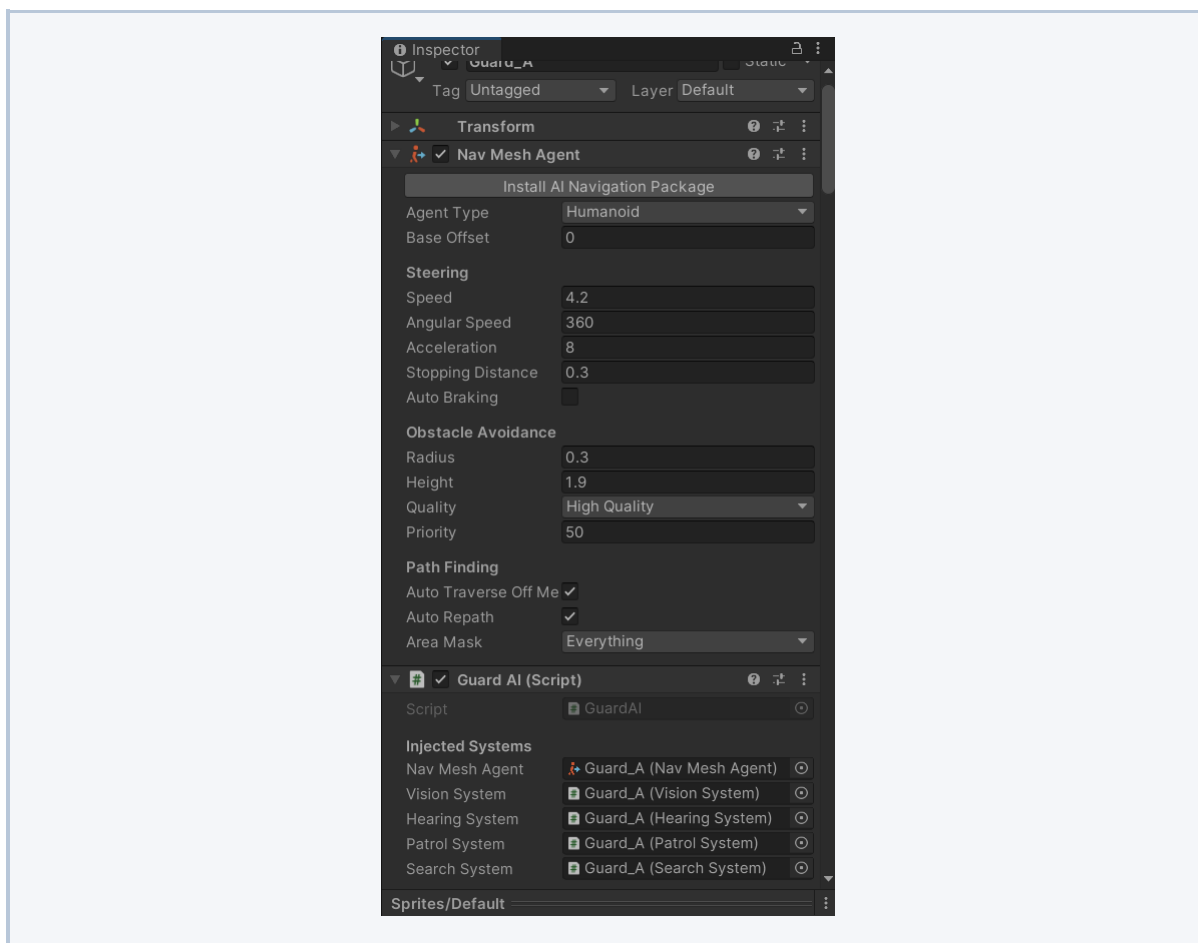
1. Играчот се движи низ нивото и се обидува да избегне детекција.
2. Guard NPC патролира низ своја зона користејќи NavMesh.
3. VisionSystem-от постепено зголемува suspicion ако играчот е во видното поле.
4. PlayerController-от создава footstep noise според начинот на движење.
5. Guard NPC реагира на виден играч или слушнат шум.
6. При целосна детекција, guard-от преминува во состојбата chasing.
7. Ако контактот се изгуби, guard-от оди до последната позната позиција и пребарува.
8. По пребарувањето, guard-от се враќа во состојбата патролирање.

5. Архитектура на AI системот

Архитектурата е поделена на мали компоненти за да биде полесно да се разбере, тестира и проширува. GuardAI е централниот координатор, но конкретните одлуки се имплементирани како behavior tree nodes и сервисни адаптери.



BehaviorTree	Основни јазли: Selector, Sequence, ConditionNode и ActionNode.
GuardAI	Главен MonoBehaviour кој ги поврзува сервисите, контекстот и behavior tree логиката.
GuardAI/*	Контекст, адаптери, интерфејси, factory и конкретни guard behavior nodes.
VisionSystem	Field-of-view, line-of-sight, suspicion value и last-known position.
HearingSystem	Глобален noise event систем кој ги известува guard listener-ите.
PatrolSystem	Избор на случајни достижни NavMesh точки за патролирање.
SearchSystem	Генерирање и посетување точки околу последната позната позиција.
LevelBuilder	Editor алатка за реконструкција на демо нивото, guard-ите, player-от и NavMesh-от.



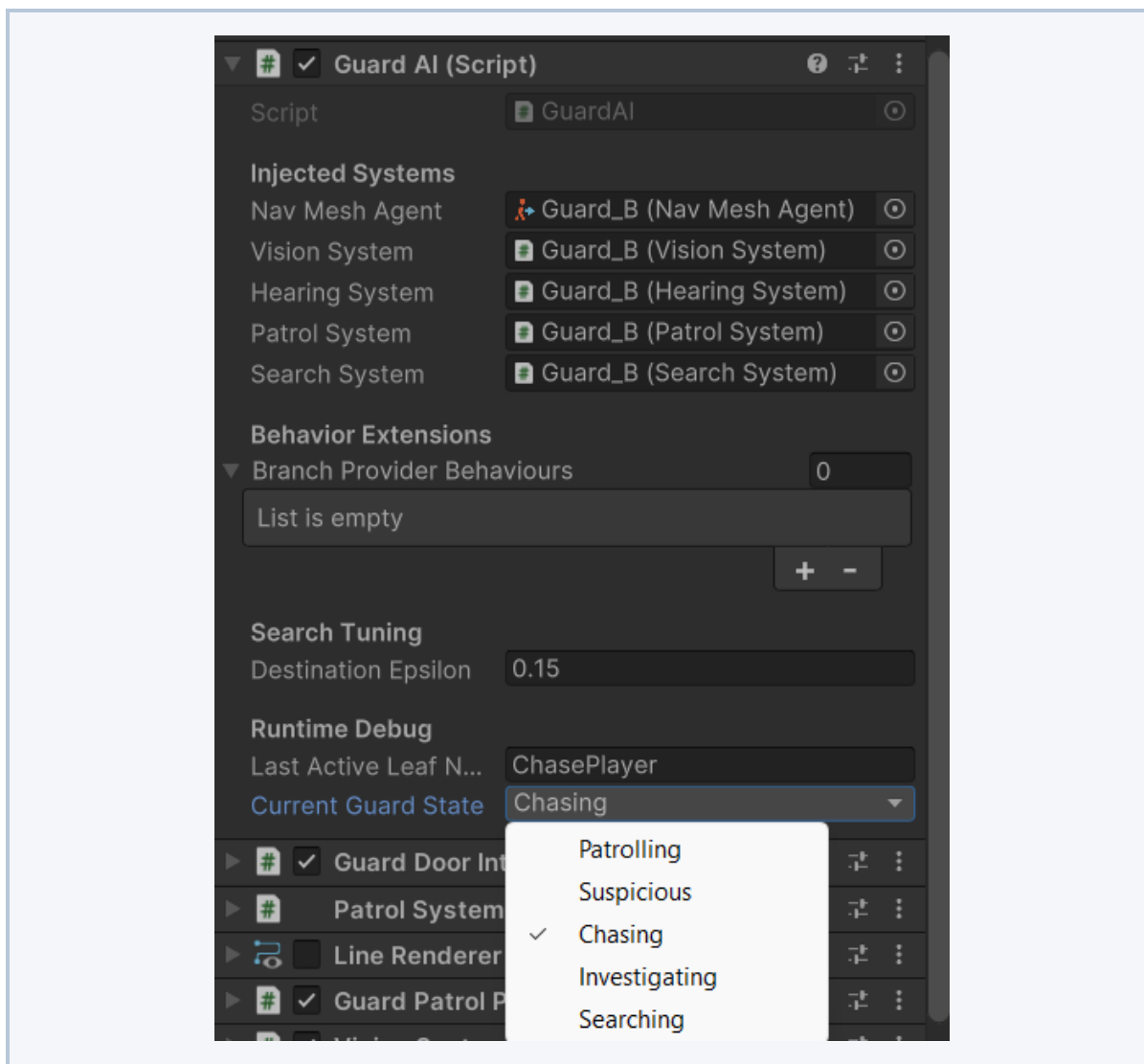
Слика: Unity Inspector со GuardAI компонентата

6. Behavior Tree

Behavior tree логиката се наоѓа во Assets/Scripts/BehaviorTree и Assets/Scripts/GuardAI. Основниот root е Selector кој ги проверува behavior branch-овите според приоритет. Ова значи дека поважните реакции, како бркање на виден играч, имаат предност пред патролирање.

Стандардниот редослед на branch-ови е:

1. Chase Sequence: ако играчот е целосно виден, guard-от го брка.
2. Suspicious Sequence: ако има делумен визуелен сигнал, guard-от се врти кон сомнителната позиција.
3. Investigate Sequence: guard-от оди до последно познатата позиција и започнува да пребарува.
4. Noise Sequence: ако guard-от слушал шум, оди до позицијата од каде дошол шумот.
5. Patrol: fallback однесување кога нема опасност или сигнал.



Слика: Screenshot за behavior tree flow



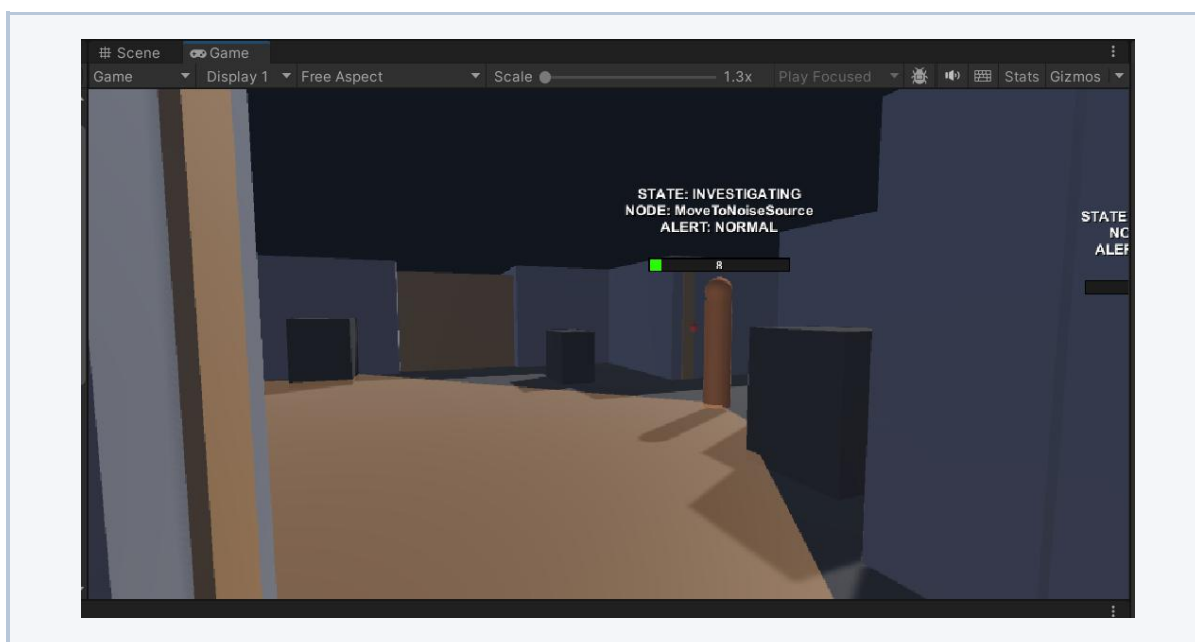
7. Guard states

Patrolling	Нормална состојба. Guard-от избира NavMesh точки и патролира.
Suspicious	Guard-от има делумен визуелен сигнал и се врти кон сомнителната позиција.
Chasing	Играчот е детектиран и guard-от го следи(брка) директно.
Investigating	Guard-от оди кон последно познатата позиција или noise source.
Searching	Guard-от пребарува повеќе точки околу последната позната позиција.

8. VisionSystem

VisionSystem е задолжен за визуелна перцепција. Системот проверува distance, field of view, line of sight и proximity awareness. Наместо моментална binary детекција, користи suspicion вредност од 0 до 100. Ова овозможува постепено забележување на играчот.

- Unaware: guard-от нема доволно сигнал.
- Suspicious: guard-от забележал нешто, но не е целосно сигурен.
- Detected: suspicion е доволно висок и guard-от има визуелен контакт.
- LastKnownPosition се зачувува кога играчот е доволно сигурно виден.



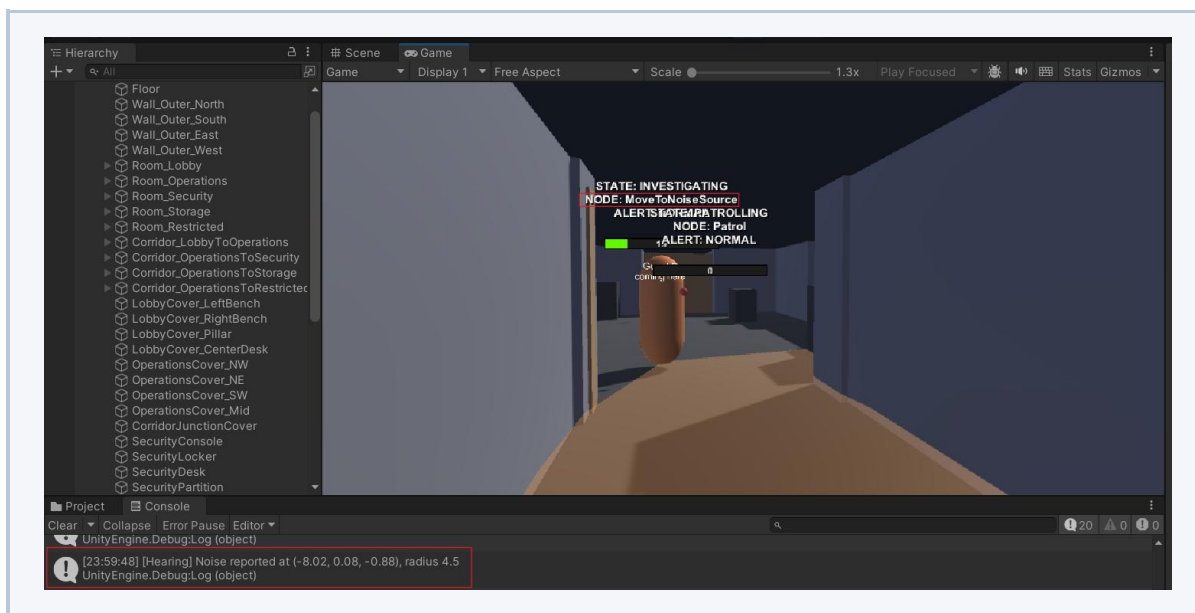
Слика: Vision cone и line-of-sight



9. HearingSystem

HearingSystem работи како глобален noise broadcaster. Кога играчот оди или трча, PlayerController повикува ReportNoise со позиција, радиус и тип на шум. Секој активен HearingSystem проверува дали е во тој радиус и, ако е, ја памети позицијата на шумот.

- Walking footstep има умерен радиус.
- Sprinting footstep има поголем радиус и полесно привлекува guard.
- Crouching има радиус 0, што го прави тивко движење.
- Шумот се памети ограничено време преку noiseDuration.

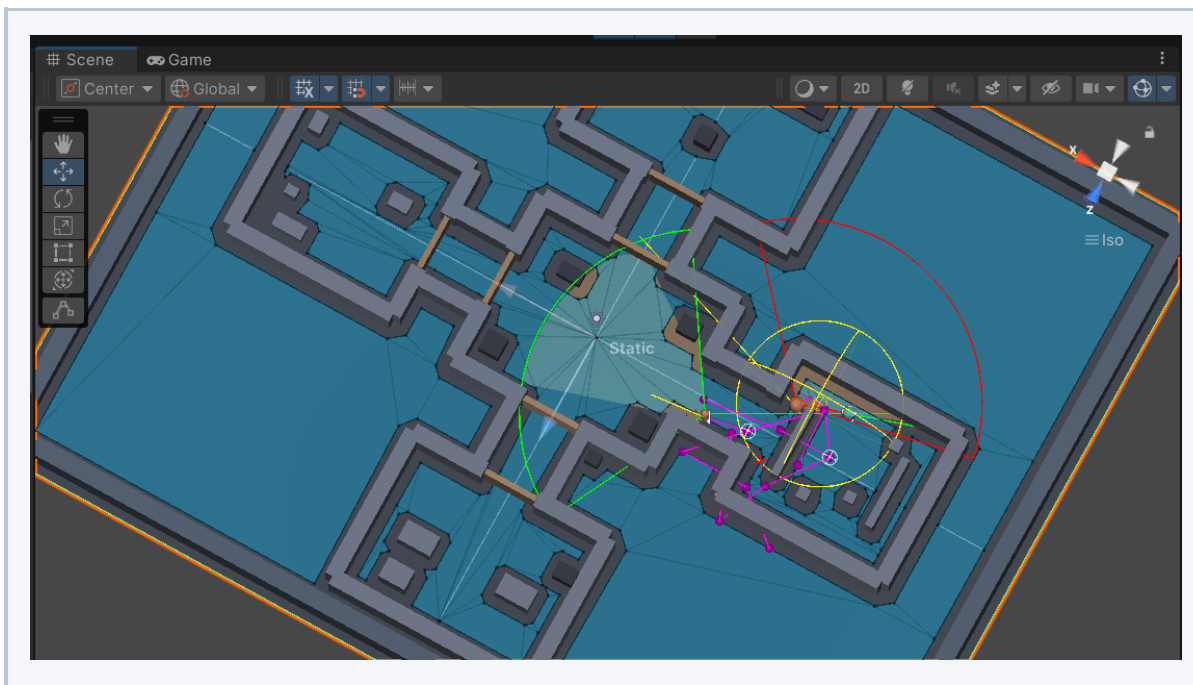


Слика: Noise investigation

10. Patrol и Search

PatrolSystem избира случајни NavMesh точки околу patrol origin. Системот проверува дали точката е достижна, дали guard-от е на NavMesh, и дали движењето не е заглавено. Ова дава природно wandering патролирање без рачно дефинирани waypoint листи.

SearchSystem се активира откако guard-от ќе стигне до последната позната позиција каде играчот бил виден или слушнат. Тој генерира повеќе точки околу таа позиција и guard-от ги посетува една по една со цел да се симулира пребарување како би бил пронајден играчот околу последно познатата позиција. Search завршува кога сите точки се посетени или кога timer-от ќе ја достигне границата.



Слика: Search points околу last-known position

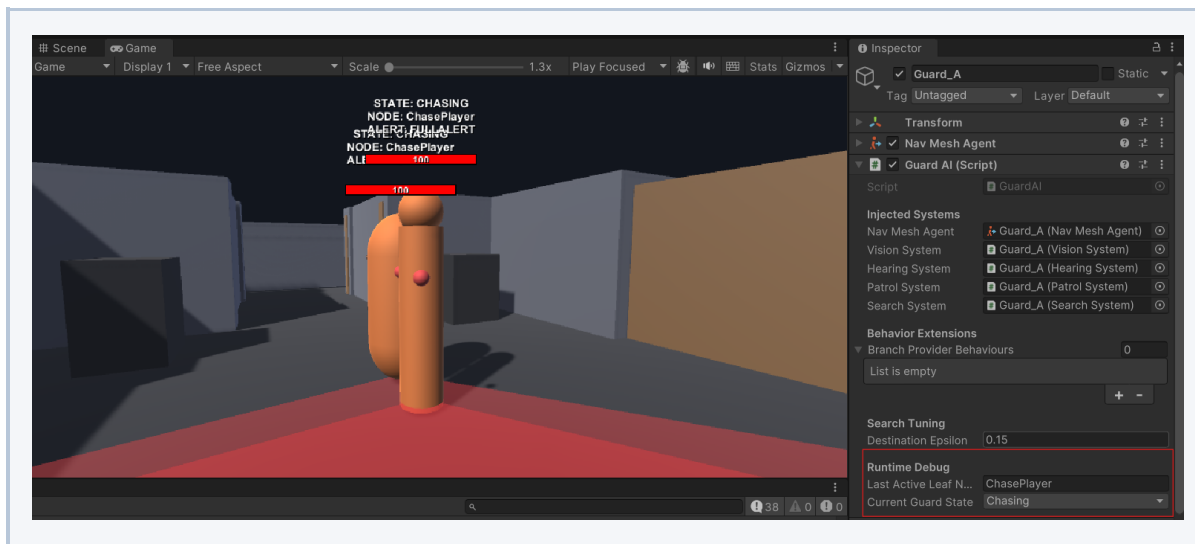
11. PlayerController

PlayerController е минимален first-person controller со movement, mouse look, crouch, sprint и footstep audio. Исто така е поврзан со HearingSystem, што значи дека движењето на играчот директно влијае врз AI реакциите.

- CharacterController се користи за движење.
- Mouse input ја ротира камерата и телото на играчот.
- Crouch ја намалува висината на CharacterController и камерата.
- Footstep clips и noise radius се подесуваат преку serialized fields.

12. GuardAlertSystem и GuardSoundSystem

GuardAlertSystem обезбедува shared alert информација меѓу guard системите, особено за позицијата каде што играчот бил виден. GuardSoundSystem додава аудио feedback за состојби како suspicious, chase, investigating, return to patrol и movement loops.

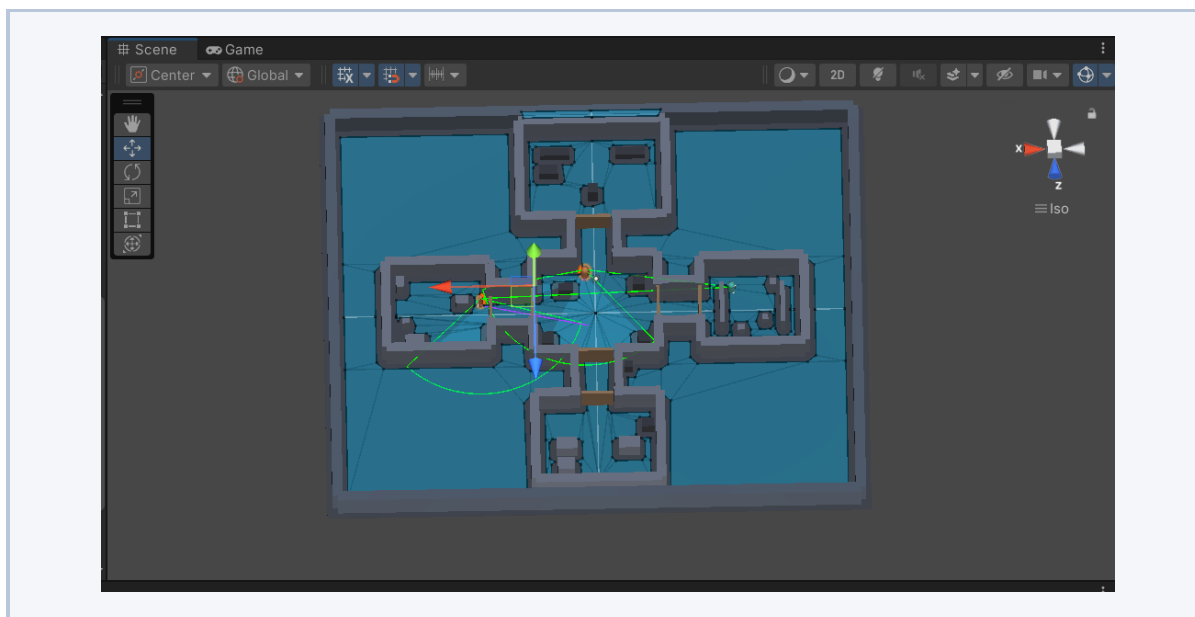


Слика: Guard state audio/debug реакција

13. Demo Level Builder

LevelBuilder.cs е Unity Editor алатка која може да изгради reproducible демо ниво. Командата Tools > Build Demo Level креира indoor layout, cover објекти, player, два guard варијанти, lighting, layers/tags, аудио поставки и NavMesh bake.

- Брише постоечки generated root пред да изгради ново ниво.
- Креира простории, коридори, cover, patrol zones и spawn позиции.
- Ги конфигурира компонентите VisionSystem, HearingSystem, PatrolSystem, SearchSystem и GuardAI.
- Автоматски поврзува audio clips од Assets/Audio ако недостасуваат.

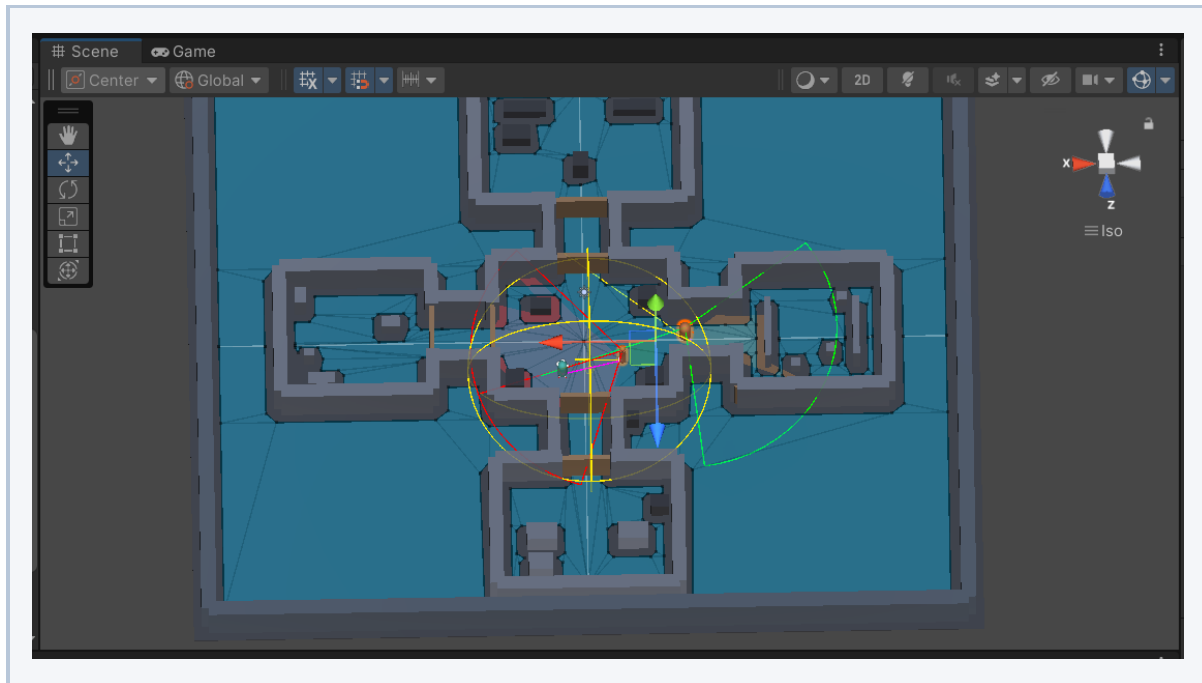


Слика: Tools > Build Demo Level

14. Debug visualization

Проектот содржи повеќе debug механизми за да се види што прави AI системот додека играта работи. Ова е важно за демонстрација, затоа што може јасно да се покаже зошто guard-от донел одредена одлука.

- GuardVisionRenderer го прикажува видното поле.
- GuardDebugVisualizer прикажува state и active node информации.
- VisionSystem црта FOV и raycast линии преку Gizmos.
- HearingSystem црта noise radius кога е слушнат шум.
- SearchSystem ги црта generated search points.



Слика: Debug визуелизација во Play Mode

15. Како може да се прошири проектот

Поради branch-provider архитектурата, guard AI системот може да се проширува без потреба од целосно менување на главната GuardAI логика. Наместо секое ново однесување да се додава директно во централната класа, проектот овозможува однесувањата да се организираат преку посебни гранки, односно IGuardBehaviorBranchProvider имплементации. На овој начин системот останува модуларен, почист и полесен за одржување.



16. Заклучок

Во рамки на проектот е изработен функционален stealth AI прототип кој демонстрира повеќе важни механики карактеристични за игри од овој жанр. Системот вклучува јасна поделба на одговорности помеѓу различните компоненти: движење на играчот, создавање звук, перцепција на чуварот, донесување одлуки преку behavior tree, движење со NavMesh, пребарување на последната позната позиција, заедничка alert состојба и debug visualization. Со ова проектот не претставува само едноставно движење на NPC, туку комплетен мал систем кој покажува како може да се организира AI логика во stealth игра.

Најважната вредност на проектот е архитектурата на guard однесувањето. Наместо целата логика да биде hard-coded во една голема скрипта, однесувањето е поделено на независни и јасно дефинирани компоненти. Ова овозможува секој дел од системот да има своја конкретна улога, како на пример услов за гледање на играчот, услов за слушање звук, акција за патролирање, акција за бркање или акција за пребарување. Ваквата поделба го прави кодот почист, поразбирлив и полесен за одржување.

Дополнително, користењето на behavior tree структура овозможува одлуките на чуварот да бидат организирани на логичен и флексибилен начин. Наместо NPC-то да реагира преку голем број неповрзани if-else услови, секоја состојба и секое однесување се дефинираат како дел од дрво на одлуки. На овој начин чуварот може да преминува помеѓу различни однесувања, како патролирање, бркање, истражување и враќање на рута, на начин кој е поедноставен за следење и проширување.

Проектот исто така покажува како може да се комбинираат повеќе Unity системи во една целина. NavMesh се користи за реалистично движење низ сцената, perception логиката овозможува чуварот да реагира на играчот и на звуци, додека debug visualization помага при тестирање и разбирање на однесувањето. Овие елементи заедно создаваат прототип кој е доволно мал за да биде разбирлив, но доволно комплетен за да прикаже реална AI структура која може да се користи како основа за поголем проект.

Како резултат, системот е лесен за тестирање, заменување и проширување. Нови услови, акции или цели гранки на однесување може да се додадат без целосно менување на постоечката GuardAI логика. Ова е особено важно кај AI за игри, бидејќи однесувањето на NPC ликовите често треба да се менува и надградува според потребите на играта.

Со ова, проектот успешно ја исполнува својата цел: да прикаже модуларен, разбирлив и функционален stealth AI систем кој ги демонстрира основните концепти на AI за игри, како перцепција, донесување одлуки, движење, реакција на играчот и организација на комплексно NPC однесување преку behavior tree архитектура.